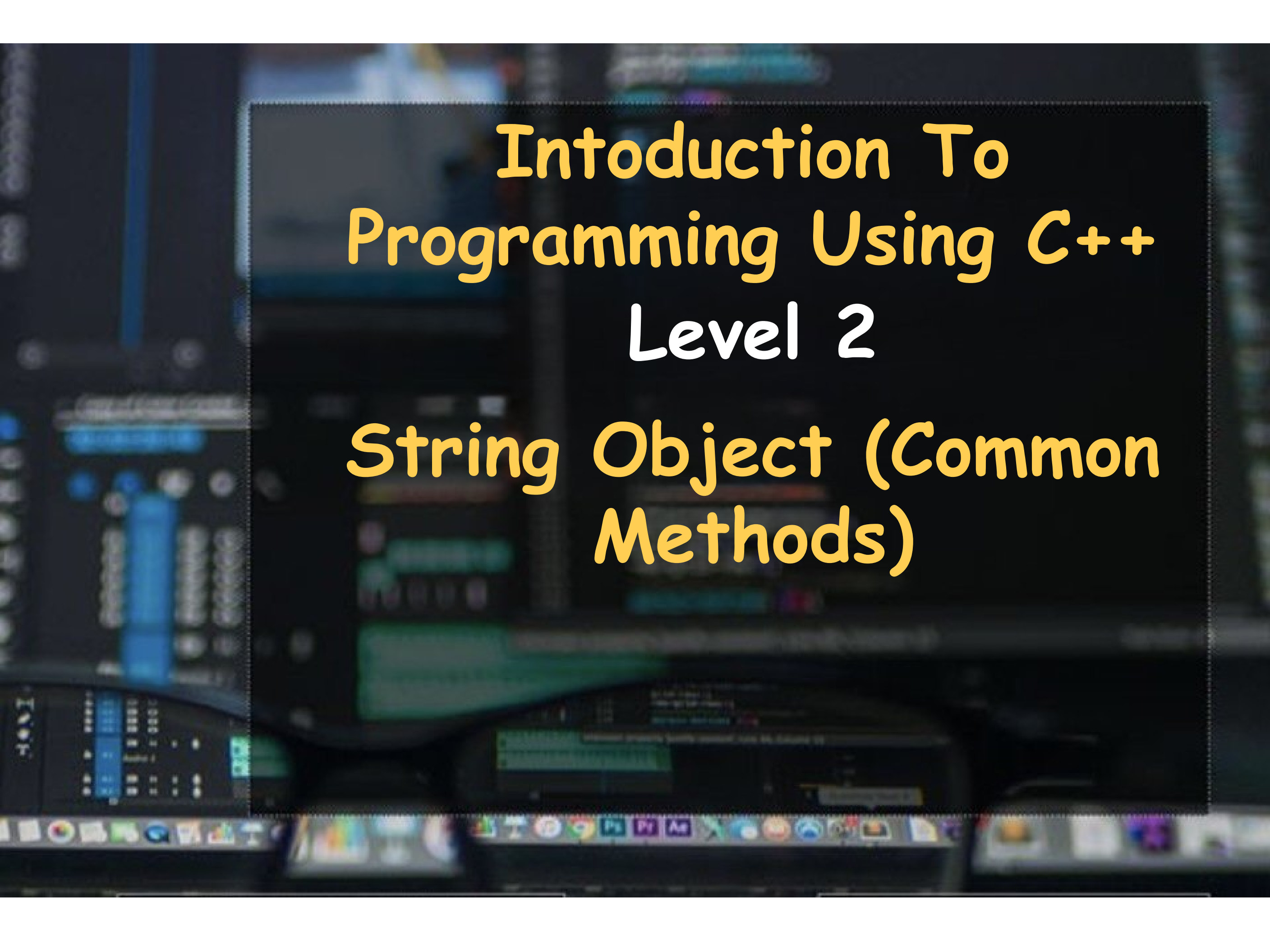
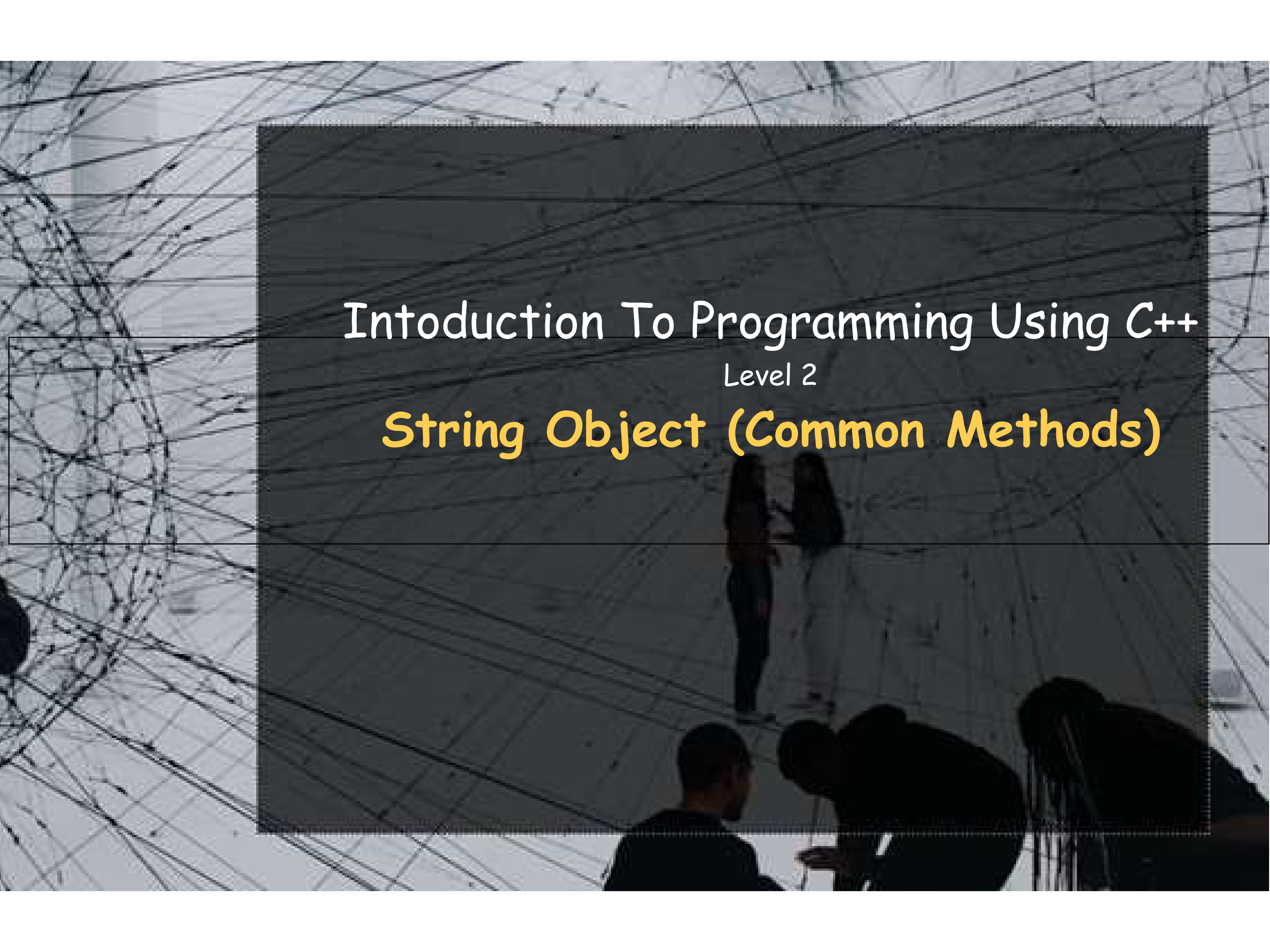


The background is a blurred image of a computer monitor. The screen shows a dark-themed code editor with some text in blue and green. At the bottom, a Windows taskbar is visible with various application icons like Chrome, Photoshop, and others.

Introduction To Programming Using C++ Level 2

String Object (Common Methods)



Introduction To Programming Using C++

Level 2

String Object (Common Methods)

🔍 Definition

■ ``string``

■ ``string`` in C++ is an object of the ``std::string`` class that represents a sequence of characters.

■ It provides many built-in methods that make working with text easier, such as adding, deleting, searching, and extracting text.

Explanation

`string` as an Object

-  `string` is a good practical example of Object-Oriented Programming (OOP).

-  When you create a `string` object, it provides methods that can be called using the dot operator (`.`).

Common Methods

-  `length()` → returns the number of characters.

-  `at(index)` → accesses a character at a specific position with bounds checking.

-  `append(text)` → adds text to the end.

-  `insert(index, text)` → inserts text at a specific position.

-  `substr(index, count)` → extracts part of the string.

-  `push\_back(char)` → adds one character to the end.

-  `pop\_back()` → removes the last character.

-  `find(text)` → searches for text and returns its position.

-  `clear()` → removes all characters.

💡 Example

```
int main()
{
    string S1 = "My Name is Mohammed Abu-Hadhoud, I Love Programming.";
    cout << S1.length() << endl;
    cout << S1.at(3) << endl;
    S1.append(" @ProgrammingAdvices");
    S1.insert(7, "Ali ");
    cout << S1.substr(16, 8) << endl;
    S1.push_back('X');
    S1.pop_back();
    cout << S1.find("Ali") << endl;
    if (S1.find("ali") == S1.npos)
    {
        cout << "ali is not found";
    }
    S1.clear();
    return 0;
}
```

➡ Result

- `S1.find("Ali")` → returns the position where `"Ali"` starts.
- `S1.find("ali")` → `std::string::npos` if `"ali"` does not exist.
- `S1.clear()` → removes all text from `S1`

Note

- ◆ `find()` is case-sensitive.
- ◆ `"Ali"` and `"ali"` are different strings.
- ◆ `std::string::npos` is a special constant used to indicate that a search did not find the requested text.
- ◆ `npos` does not simply mean "a very large number"; it is the special value returned by string search functions when the requested position cannot be found.

⊘ Warning

- ◆ Using `[]` to access a character does not perform bounds checking.
- ◆ `at()` performs bounds checking and throws an exception when the index is invalid.
- ◆ Be careful when using the result of `find()` as an index, because `find()` may return `std::string::npos`.



Comparison

▨ `at(index)`

- ✓ Performs bounds checking.
- ✓ Throws `std::out\_of\_range` if the index is invalid.
- ✓ Useful when safer access is required.

▤ `operator[]`

- ✓ Does not perform bounds checking.
- ✓ Invalid index results in undefined behavior.
- ✓ Useful when the index is known to be valid.

✦ Summary

- ✓ `std::string` is an object used to work with text in C++.
- ✓ `length()` returns the number of characters.
- ✓ `at()` accesses a character with bounds checking.
- ✓ `append()` adds text to the end.
- ✓ `insert()` inserts text at a specific position.
- ✓ `substr()` extracts part of a string.
- ✓ `push_back()` adds one character.
- ✓ `pop_back()` removes the last character.
- ✓ `find()` searches for text.
- ✓ `npos` indicates that the searched text was not found.
- ✓ `clear()` removes all characters from the string.

Thank you 😊